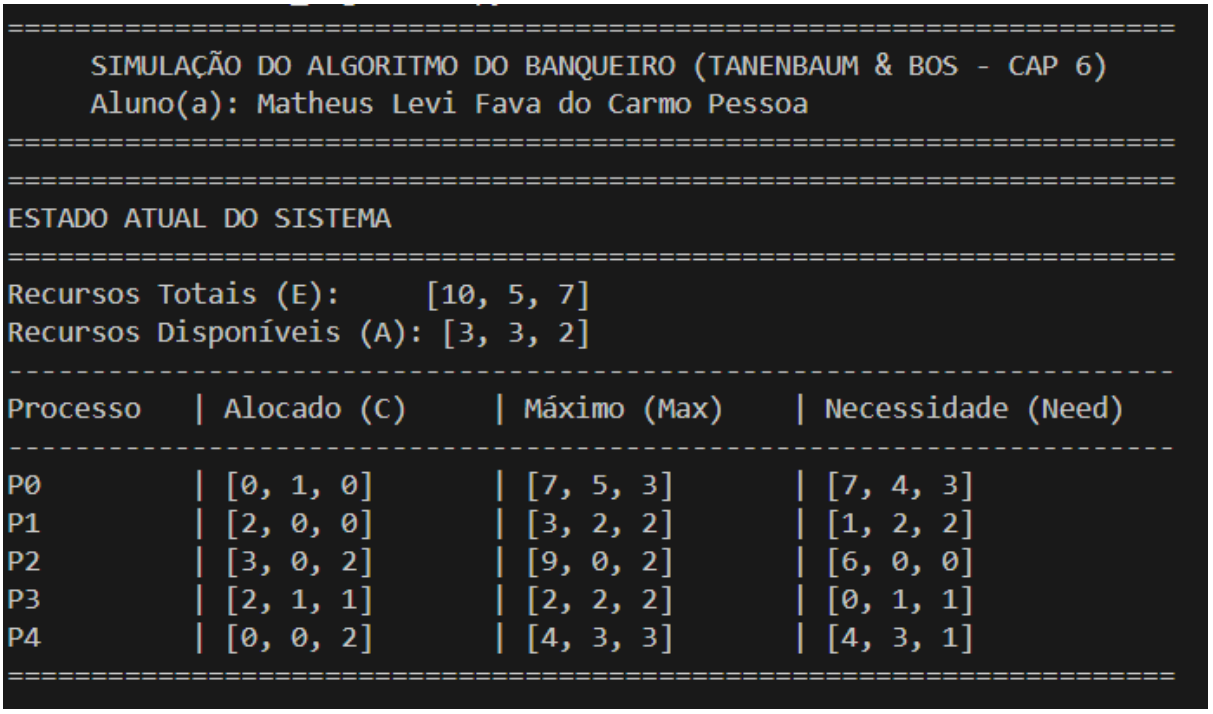
Aluno: Matheus Levi Fava do Carmo Pessoa
Disciplina: Sistemas Operacionais
Atividade 07

Teste 1: Estado inicial dos processos e recursos



```
======================================================================
     SIMULAÇÃO DO ALGORITMO DO BANQUEIRO (TANENBAUM & BOS - CAP 6)
     Aluno(a): Matheus Levi Fava do Carmo Pessoa
======================================================================

======================================================================
ESTADO ATUAL DO SISTEMA
======================================================================
Recursos Totais (E):     [10, 5, 7]
Recursos Disponíveis (A): [3, 3, 2]
----------------------------------------------------------------------
Processo   | Alocado (C)     | Máximo (Max)    | Necessidade (Need)
----------------------------------------------------------------------
P0         | [0, 1, 0]       | [7, 5, 3]       | [7, 4, 3]
P1         | [2, 0, 0]       | [3, 2, 2]       | [1, 2, 2]
P2         | [3, 0, 2]       | [9, 0, 2]       | [6, 0, 0]
P3         | [2, 1, 1]       | [2, 2, 2]       | [0, 1, 1]
P4         | [0, 0, 2]       | [4, 3, 3]       | [4, 3, 1]
======================================================================
```

Nesta primeira etapa, o sistema carrega a configuração inicial dos processos e constrói as matrizes essenciais do algoritmo. Temos o vetor de Recursos Totais (E) com 10 unidades de R0, 5 de R1 e 7 de R2. A matriz de Alocação (C) indica a quantidade de instâncias que já estão retidas pelos processos (P0 a P4), enquanto a matriz Máxima (Max) representa o teto de requisição que cada processo exige para concluir sua execução.

A matriz de Necessidade (Need) é calculada dinamicamente subtraindo a alocação da matriz máxima (Need = Max - C), indicando o quanto falta para cada processo. Por fim, o vetor de Recursos Disponíveis (A) é obtido subtraindo o total já alocado (C) dos recursos totais em estoque (E), o que resulta no saldo inicial livre de [3, 3, 2].

Teste 2: Verificação do estado seguro

```
--------------------------------------------------------------------------
[VERIFICAÇÃO DE ESTADO SEGURO]
ALUNO - Matheus Levi Fava do Carmo Pessoa
--------------------------------------------------------------------------
Vetor de Trabalho Inicial (Work/Disponível): [3, 3, 2]

Passo 1: P1 pode ser atendido!
  Necessidade [1, 2, 2] <= Disponível [3, 3, 2]
  P1 executa e libera seus recursos alocados: [2, 0, 0]
  Novo Vetor de Trabalho (Work): [5, 3, 2]

Passo 2: P3 pode ser atendido!
  Necessidade [0, 1, 1] <= Disponível [5, 3, 2]
  P3 executa e libera seus recursos alocados: [2, 1, 1]
  Novo Vetor de Trabalho (Work): [7, 4, 3]

Passo 3: P0 pode ser atendido!
  Necessidade [7, 4, 3] <= Disponível [7, 4, 3]
  P0 executa e libera seus recursos alocados: [0, 1, 0]
  Novo Vetor de Trabalho (Work): [7, 5, 3]

Passo 4: P2 pode ser atendido!
  Necessidade [6, 0, 0] <= Disponível [7, 5, 3]
  P2 executa e libera seus recursos alocados: [3, 0, 2]
  Novo Vetor de Trabalho (Work): [10, 5, 5]

Passo 5: P4 pode ser atendido!
  Necessidade [4, 3, 1] <= Disponível [10, 5, 5]
  P4 executa e libera seus recursos alocados: [0, 0, 2]
  Novo Vetor de Trabalho (Work): [10, 5, 7]

✅ SUCESSO: O sistema está em um ESTADO SEGURO!
   Sequência Segura de Execução: P1 -> P3 -> P0 -> P2 -> P4
```

Neste teste, o algoritmo valida se o estado inicial do sistema é seguro antes de processar qualquer nova requisição. Utilizando um vetor de trabalho (Work) iniciado com os recursos disponíveis [3, 3, 2], o sistema busca iterativamente algum processo cuja necessidade pendente possa ser atendida.

O processo P1 é o primeiro a ser selecionado, pois sua necessidade [1, 2, 2] é menor ou igual ao vetor Work. O algoritmo simula sua execução até o fim, momento em que P1 libera seus recursos retidos [2, 0, 0], atualizando o Work para [5, 3, 2]. O fluxo segue repetindo essa lógica e atendendo sequencialmente P3, P0, P2 e P4. Como todos os processos conseguem ser simulados até a conclusão, fica provado matematicamente que o sistema não possui risco de impasses (deadlocks), confirmando que o estado inicial é seguro.

Teste 3: Solicitação de recursos pelo processo P1

```
SOLICITAÇÃO DE RECURSOS PELO PROCESSO P1: [1, 0, 2]
ALUNO - Matheus Levi Fava do Carmo Pessoa
########################################################################
⚠️TENTATIVA PROVISÓRIA: Recursos disponíveis no momento. Testando alocação...

------------------------------------------------------------------------
[VERIFICAÇÃO DE ESTADO SEGURO]
ALUNO - Matheus Levi Fava do Carmo Pessoa
------------------------------------------------------------------------
Vetor de Trabalho Inicial (Work/Disponível): [2, 3, 0]

Passo 1: P1 pode ser atendido!
  Necessidade [0, 2, 0] <= Disponível [2, 3, 0]
  P1 executa e libera seus recursos alocados: [3, 0, 2]
  Novo Vetor de Trabalho (Work): [5, 3, 2]

Passo 2: P3 pode ser atendido!
  Necessidade [0, 1, 1] <= Disponível [5, 3, 2]
  P3 executa e libera seus recursos alocados: [2, 1, 1]
  Novo Vetor de Trabalho (Work): [7, 4, 3]

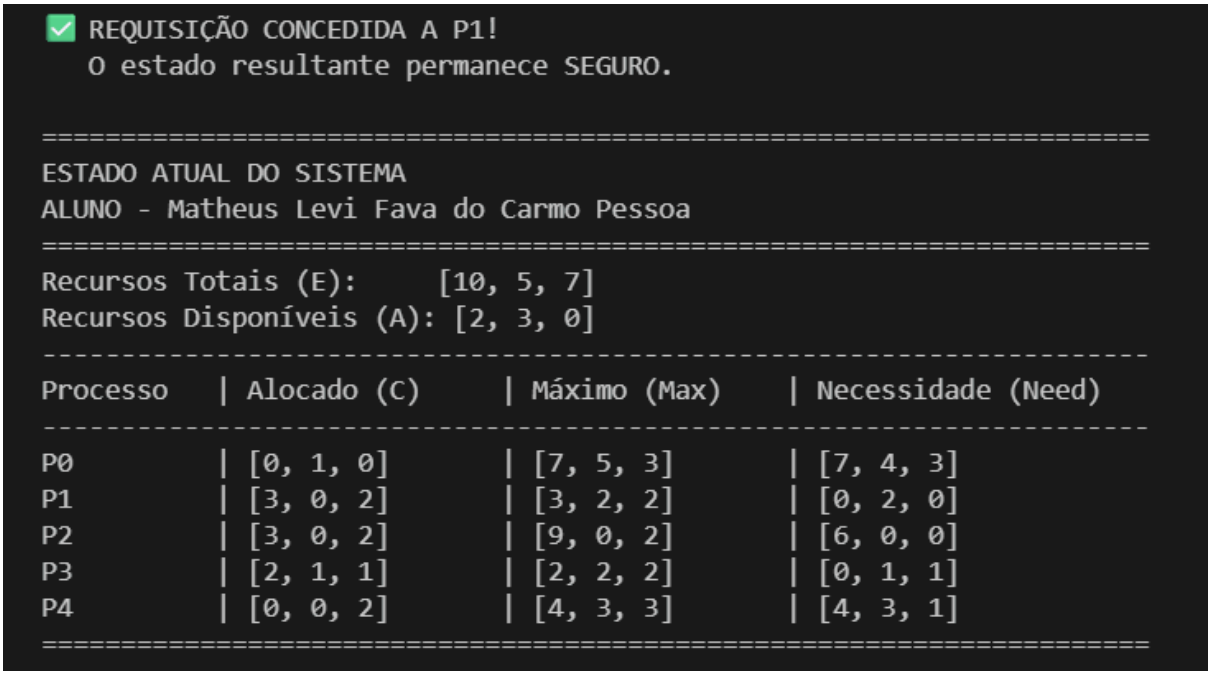
Passo 3: P0 pode ser atendido!
  Necessidade [7, 4, 3] <= Disponível [7, 4, 3]
  P0 executa e libera seus recursos alocados: [0, 1, 0]
  Novo Vetor de Trabalho (Work): [7, 5, 3]

Passo 4: P2 pode ser atendido!
  Necessidade [6, 0, 0] <= Disponível [7, 5, 3]
  P2 executa e libera seus recursos alocados: [3, 0, 2]
  Novo Vetor de Trabalho (Work): [10, 5, 5]

Passo 5: P4 pode ser atendido!
  Necessidade [4, 3, 1] <= Disponível [10, 5, 5]
  P4 executa e libera seus recursos alocados: [0, 0, 2]
  Novo Vetor de Trabalho (Work): [10, 5, 7]

✅ SUCESSO: O sistema está em um ESTADO SEGURO!
   Sequência Segura de Execução: P1 -> P3 -> P0 -> P2 -> P4

✅ REQUISIÇÃO CONCEDIDA A P1!
   O estado resultante permanece SEGURO.
```

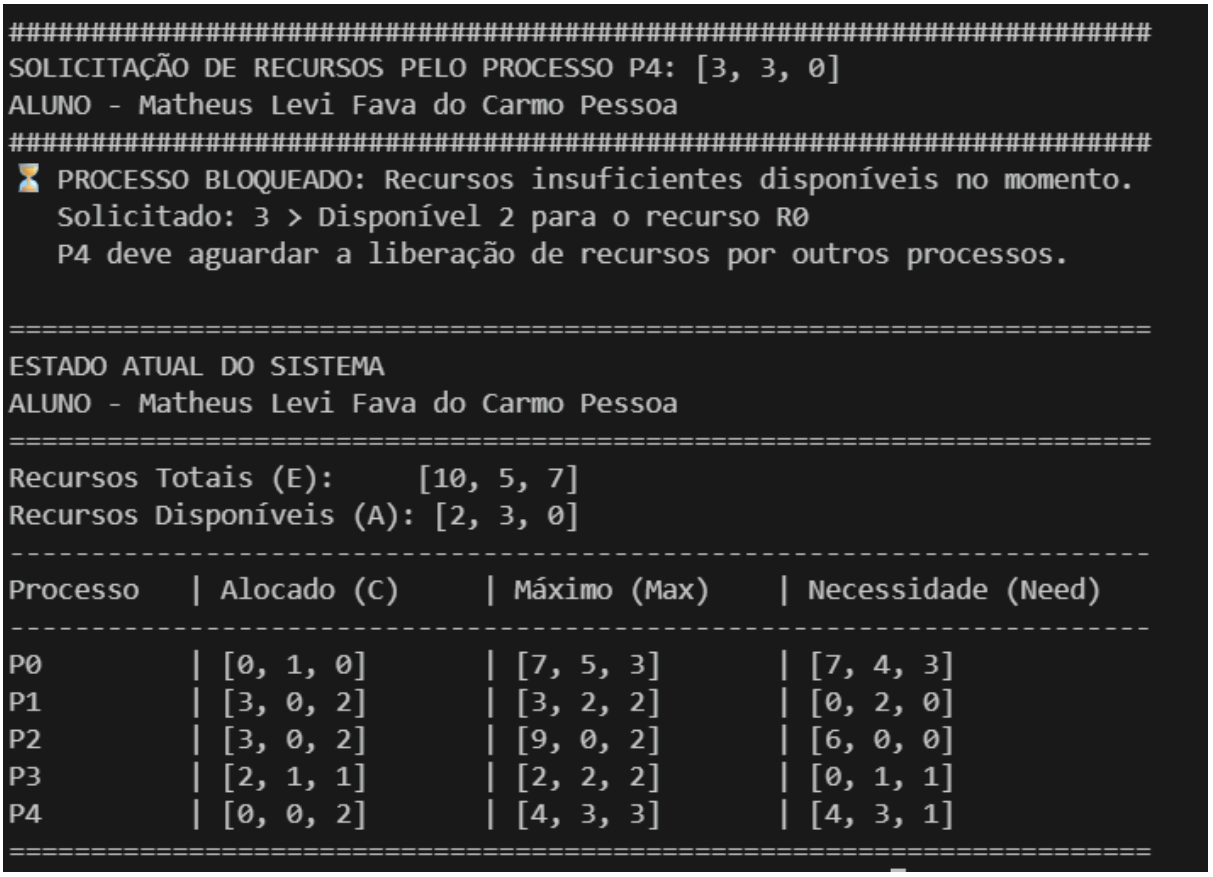


Neste cenário, o processo P1 realiza uma requisição de recursos solicitando o vetor [1, 0, 2]. O algoritmo executa duas validações cruciais de segurança:

> Verifica se o pedido não excede a necessidade máxima declarada pelo processo ([1, 0, 2] <= [1, 2, 2]). Condição satisfeita.
>
> Verifica se a quantidade solicitada está imediatamente disponível no sistema ([1, 0, 2] <= [3, 3, 2]). Condição satisfeita.

Após passar nas validações, o sistema realiza uma alocação provisória, reduzindo os recursos disponíveis para [2, 3, 0], e executa novamente a rotina de verificação de estado seguro. Como o estado resultante se mantém seguro e todos os processos ainda conseguem finalizar, a requisição é definitivamente concedida. A matriz de alocação de P1 é então atualizada para [3, 0, 2] e sua matriz de necessidade cai para [0, 2, 0].

Teste 4: Solicitação de recursos pelo processo P4



Por fim, o processo P4 tenta realizar uma requisição solicitando o vetor [3, 3, 0] (3 unidades de R0 e 3 de R1). O primeiro critério de segurança passa, pois o pedido está estritamente dentro da necessidade máxima restante do processo ([3, 3, 0] <= [4, 3, 1]).

No entanto, na segunda etapa de validação (Request <= Available), o algoritmo detecta que os recursos livres atuais no sistema são apenas [2, 3, 0]. Como o processo P4 exige 3 unidades do recurso R0, mas só existem 2 disponíveis em estoque no momento, a requisição precisa ser negada. Como medida de prevenção de deadlocks, P4 é bloqueado e colocado em estado de espera até que outros processos finalizem e liberem unidades suficientes de R0.